

Introduction and Requirement engineering

Chapter 1 and 2

Introduction

Software is a computer program that follows defined procedural steps or algorithms to perform specific tasks.

Software engineering is the disciplined application of scientific principles to design, develop, document, operate, and maintain software systems.

Software products are complete software systems delivered with documentation explaining installation and use. They can be:

- **Generic products:** designed for the general market (e.G., Word processors, databases).

- **Customized products:** developed for a specific client or purpose (e.G., Business process systems, control systems).

A **process** is a structured set of activities, actions, and tasks used to create a work product.

Software Application Domains

- 1.System software:** Supports other programs (e.g., compilers, operating systems, network software).
- 2.Application software:** Designed for end-user tasks.
- 3.Engineering/scientific software:** Used for technical and scientific computations.
- 4.Embedded software:** Built into devices to control their functions (e.g., car fuel control, braking).
- 5.Product-line software:** Developed for a family of related products.
- 6.Web/mobile applications:** Software designed for online or mobile use.
- 7.Artificial intelligence software:** Uses heuristics and learning techniques for complex problem-solving (e.g., robotics, decision systems, pattern recognition, game playing).

Software process

A **software process** is a set of related activities that results in the creation of a software product.

All software processes share four **fundamental activities**:

1.Specification (Requirements Engineering): Defining what the software should do and the constraints on its operation.

2.Development (Design and Implementation): Designing the structure and writing the code for the software.

3.Validation: Ensuring the developed software meets customer requirements.

4.Evolution: Modifying the software to adapt to new customer or market needs.

These activities are organized differently depending on the **development model** used:

- In the **Waterfall model**, they occur in a strict sequence.
- In **Incremental development**, they are performed in overlapping or iterative cycles.

The execution of these activities also varies according to the **type of software**, the **team members**, and the **organizational structure** involved.

Software Requirements and Requirements Engineering

System requirements describe what a system should do, the services it must provide, and the constraints under which it operates. These requirements express the **customers' needs** for a system designed to achieve specific goals, such as controlling devices, processing orders, or retrieving information.

Requirements engineering (software specification) is the process of **understanding, defining, and documenting** the required services and operational constraints of the system.

It includes four main activities:

- 1. Feasibility study:** Assessing whether the proposed system is practical and achievable.
- 2. Requirements elicitation and analysis:** Gathering and examining user needs and expectations.
- 3. Requirements specification:** Formally documenting the system's functions and constraints.
- 4. Requirements validation:** Ensuring the documented requirements accurately represent user needs.

Types of Software Requirements

Software requirements describe what a system should do and the conditions under which it operates. They are categorized as follows:

1. User Requirements

- High-level statements describing the **services** the system should provide and the **constraints** it must follow.
- Focus only on the **external behavior** of the system.
- Written in **natural language**, using simple tables or diagrams (not code or formal notation).
- Used mainly by **clients, end-users, and system architects**.

2. System Requirements

- More detailed descriptions of the **functions, services, and operational constraints** of the software.
- Do **not** specify how the system will be designed or implemented.
- Used by **developers, engineers, and system architects** for system construction.

Types of Software Requirements

Classification by Nature

A. Functional Requirements

- Define **what services or functions** the system should perform, how it reacts to inputs, and how it behaves in specific situations.
- Depend on the **software type**, **user expectations**, and **organizational standards**.

B. Non-Functional Requirements

- Define **constraints** on system operation, such as **performance**, **timing**, **standards**, or **development limitations**.
- Usually apply to the **entire system**, not individual components.
- More challenging to link directly to components because they often affect the **overall architecture** (e.g., minimizing communication to improve performance).

Relationship Between Non-Functional and Functional Requirements

A **single non-functional requirement** (for example, **security**) can lead to:

- **New functional requirements** — additional system services needed to meet that non-functional goal (e.g., user authentication).
- **Restrictions on existing requirements** — limiting how current system functions operate to ensure compliance with the non-functional constraint.

Requirements Engineering Activities

* Feasibility Study

A **feasibility study** estimates whether user needs can be met using existing software and hardware technologies. It evaluates if the proposed system is **technically possible, cost-effective, and achievable within budget and time constraints.**

The study should be **quick and inexpensive**, and its outcome determines whether to proceed with detailed system analysis.

Requirements Elicitation and Analysis

Requirements Elicitation and Analysis This process identifies **user and system requirements** by defining the application domain, required services, performance expectations, and system constraints.

Stakeholders include everyone who influences requirements—such as **end-users, engineers, managers, domain experts, and union representatives**.

The process consists of four main activities:

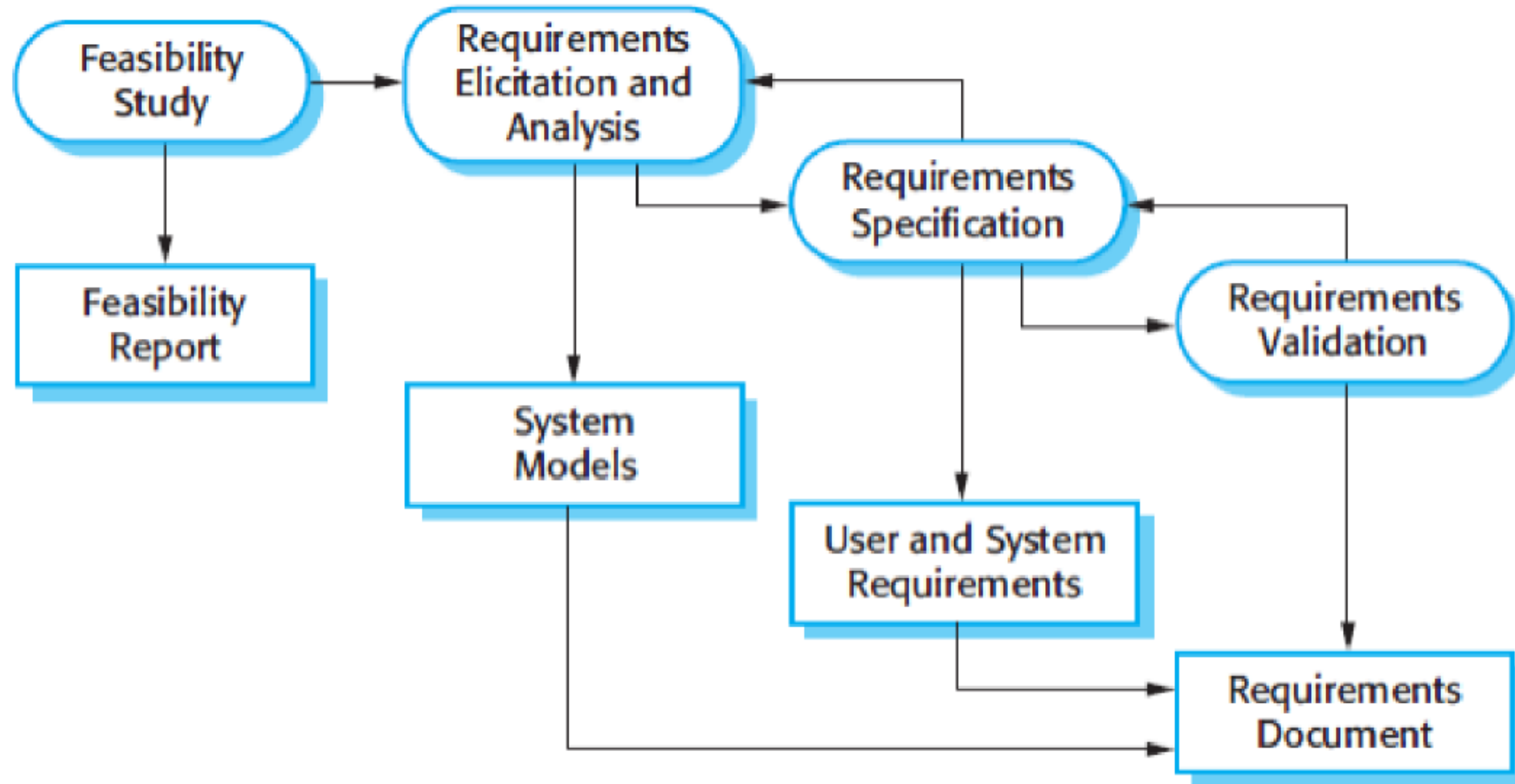
1.Requirements Discovery (Elicitation):

Collecting information about the desired system and existing systems through **documents, stakeholder interaction, scenarios, and prototypes** to clarify needs.

2.Requirements Classification and Organization:

Grouping and structuring related requirements into **coherent clusters**—often aligned with the **system's architecture** and sub-systems.

Requirement engineering process (exam)



Requirements Elicitation and Analysis

1.Requirements Prioritization and Negotiation:

Handling **conflicts among stakeholders** by prioritizing and reconciling differences through **discussion and compromise**.

2.Requirements Specification:

Documenting and formalizing the discovered and agreed-upon requirements for use in subsequent phases.

Principles guiding analysis methods:

- 1.Understand and represent the **information domain**.
- 2.Define the **functions** the software performs.
- 3.Represent the **behavior** in response to external events.
- 4.Partition models into **layers or hierarchies** to show increasing detail.
- 5.Move progressively from **essential information** to **implementation details**.

Requirements Specification

This activity involves translating analyzed information into a **Software Requirements Specification (SRS)** — a formal document describing both **user and system requirements**.

The requirements must be **clear, unambiguous, complete, consistent, and easy to understand**.

The **SRS document** serves as an official agreement and may combine or separate user and system requirements depending on complexity.

Used by:

- 1.System customers** – to confirm the system meets their needs.
- 2.Managers** – to plan bids and manage project development.
- 3.System engineers** – to understand what system to build.
- 4.Test engineers** – to design validation tests.
- 5.Maintenance engineers** – to understand the system's structure and relationships.

Requirements Validation

Requirements validation is the process of reviewing gathered, analyzed, and documented requirements to ensure their **accuracy, completeness, and feasibility**. It is essential because **errors in the requirements document** can cause major rework and high costs if discovered during or after system development.

During validation, several types of **checks** are performed:

- 1. Validity check:** Ensures the system functions truly reflect user needs; additional or revised functions may be identified.
- 2. Consistency check:** Confirms that no requirements conflict or contradict each other.
- 3. Completeness check:** Verifies that all intended functions and constraints are included.
- 4. Realism check:** Ensures requirements are **practical and achievable** within current technology, budget, and schedule limits.
- 5. Verifiability check:** Confirms each requirement can be **tested and verified**, preventing future disputes between the customer and developer.

Requirements Modeling

Requirements modeling involves creating visual representations (models) that help software engineers understand and define what the system should do.

These models describe the **data**, **functions**, and **behavior** of the system.

Types of Models:

1.Data Models:

Represent the **information domain** of the problem — how data are stored and related.

Example: **Entity Relationship Diagram (ERD).**

2.Functional Models:

Show how the software **processes data** through **input, processing, and output** operations.

Examples: **Data Flow Diagram (DFD)** and **Control Flow Diagram (CFD).**

3.Behavioral Models:

Describe how the software **responds to external events** and **changes its state** (e.g., waiting, computing, or printing).

Example: **State Transition Diagram (STD).**

Requirements Modeling

Objectives of Requirements Modeling:

- 1.To clearly describe **what the customer requires**.
- 2.To provide a **foundation for software design** (architecture, interface, components).
- 3.To define a **set of requirements** that can be **validated** after the software is built.

Requirements Management

Requirements management is the process of **tracking, controlling, and updating** system requirements as they evolve.

In large software systems, requirements **change frequently** because all problems cannot be fully defined at the beginning. After the **initial requirements** are set, stakeholders often introduce **new or modified requirements**.

The engineering team must:

- Add and integrate new requirements with existing ones.
- Update relationships among requirements to maintain consistency.
- Establish a **formal change-control process** to evaluate, approve, and document modifications and link them to the corresponding system requirements.

Thank you

Design and Implementation

Design Modeling in Software Engineering

Design Modeling in Software Engineering

Component level design —

User Interfaces design —

Architectural design —

Data design —



A good Design

- 1- A good design must fulfill all **explicit and implicit requirements** in the requirements model to ensure stakeholder satisfaction.
- 2- It should serve as a **clear, readable, and understandable guide** for developers, testers, and maintenance teams throughout the software lifecycle.
- 3- The design must provide a **comprehensive representation** of **data, functional, and behavioral aspects**.

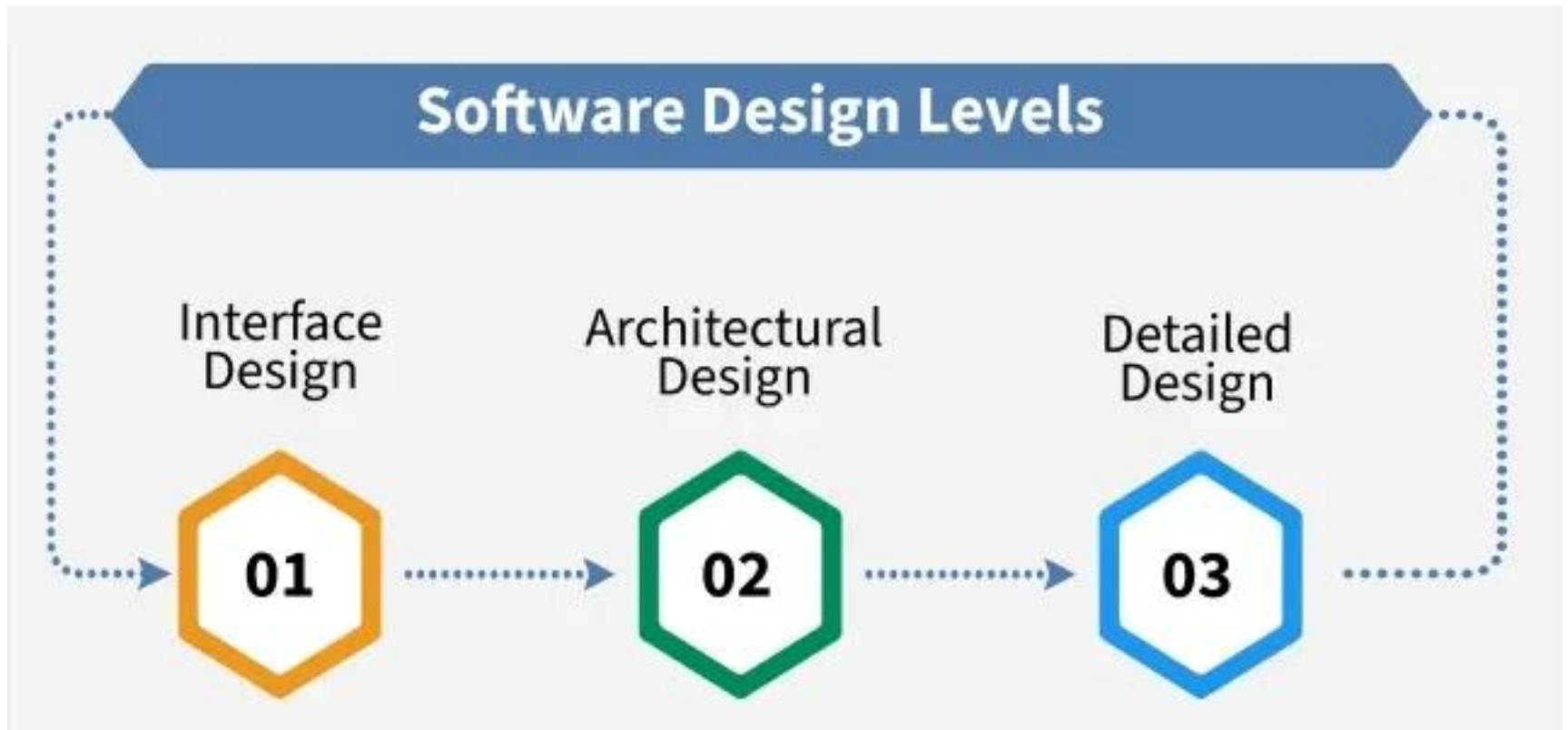
Data Design

- Data design defines how information is organized and related within a system. It uses Entity-Relationship Diagrams (ERDs) to represent data objects (entities) and their connections. Each entity contains specific information, and relationships describe how entities interact. The data structure is often visualized as tables with three main types of relationships: **one-to-one**, **one-to-many**, and **many-to-many**, indicating different ways entities are linked to each other.

Software Design Process

- **Software Design Process** is the phase where developers plan how to turn a set of requirements into a working system. Like a blueprint for the software. Instead of going straight into writing code, developers break down complex requirements into smaller, manageable pieces, design the system architecture, and decide how everything will fit together and work.
- **In short sentences :**
- **Like drawing a map before starting a trip.**
- **Developers plan how the system will work before coding.**
- **A good design saves time and prevents confusion.**
- The software design process can be divided into the following three levels or phases of design:

Software Design Levels



Interface Design

Interface design defines how a system interacts with its external environment, focusing on the communication between the system and its users, devices, or other systems. At this stage, the system is viewed as a black box, meaning its internal operations are ignored. The design emphasizes interaction and dialogue rather than internal implementation, ensuring smooth and effective communication across all interfaces.

- **In short sentences :**

Defines how people or systems talk to your software.

Example: When a customer clicks 'Order Now', the system knows what to do.

Interface Design

Interface design should include the following details:

- **Precise description** of events in the environment, or messages from agents to which the system must respond.
- **Precise description** of the events or messages that the system must produce.
- **Specification** of the data, and the formats of the data coming into and going out of the system.
- **Specification** of the ordering and timing relationships between incoming events or messages, and outgoing events or outputs.

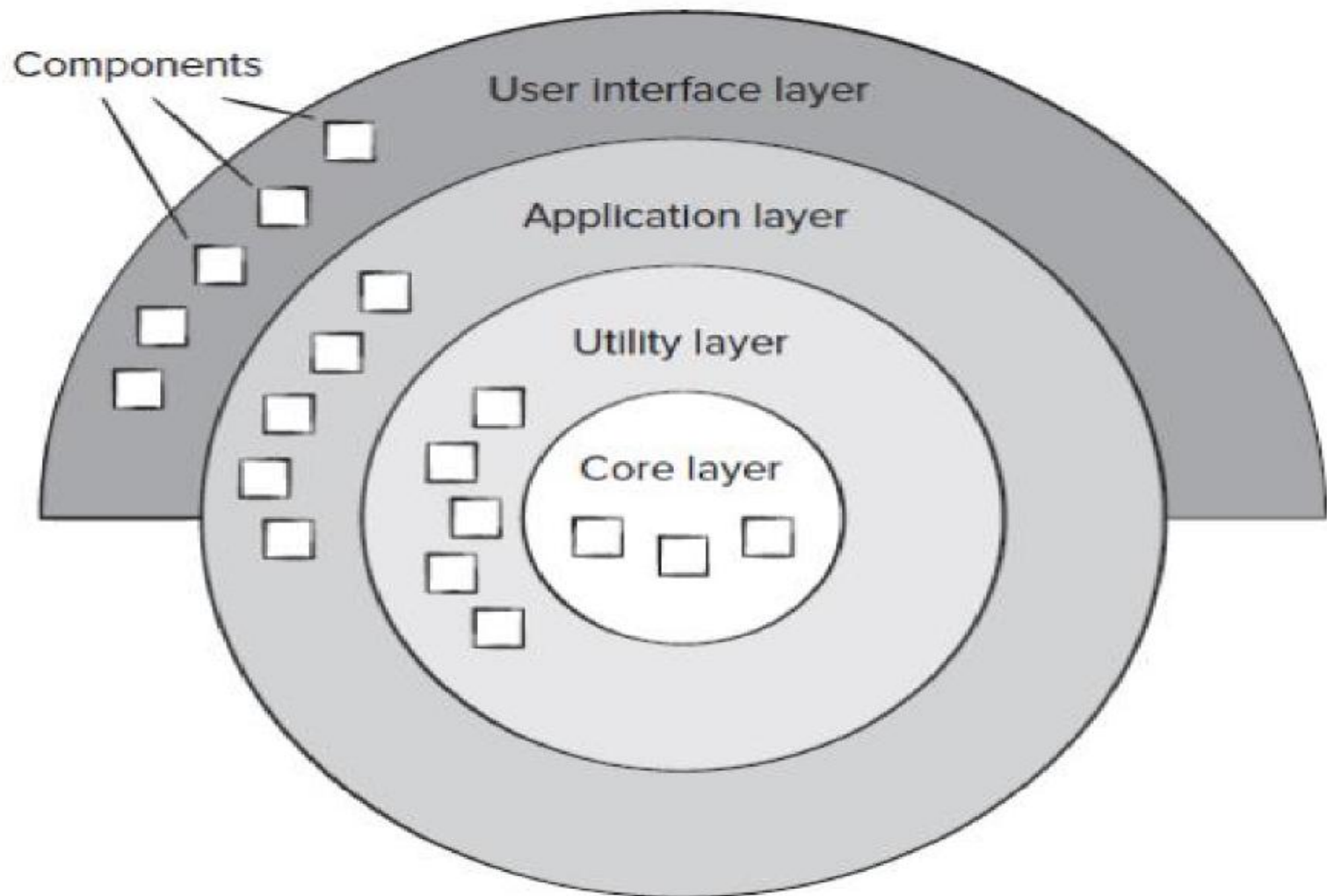
Architectural Design

- **Architectural design** is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them. In architectural design, the overall structure of the system is chosen, but the internal details of major components are ignored.
- **In short sentences:**
- Shows the big picture of how the main parts connect.
- Example: App → Backend → Database → Delivery.

Layered Architecture

- The system is divided into layers, each with a specific level of functionality.
- Each layer provides services to the layer above it and uses services from the layer below.
- The core layer handles low-level operations, while the UI layer manages user interaction.
- Supports multi-level security, modular development, and easy maintenance.
- **Advantages:**
 - Layers can be replaced or modified independently.
 - Enhances reliability and maintainability.
- **Disadvantages:**
 - Difficult to keep clean separation between layers.
 - Upper layers may depend on lower layers directly.

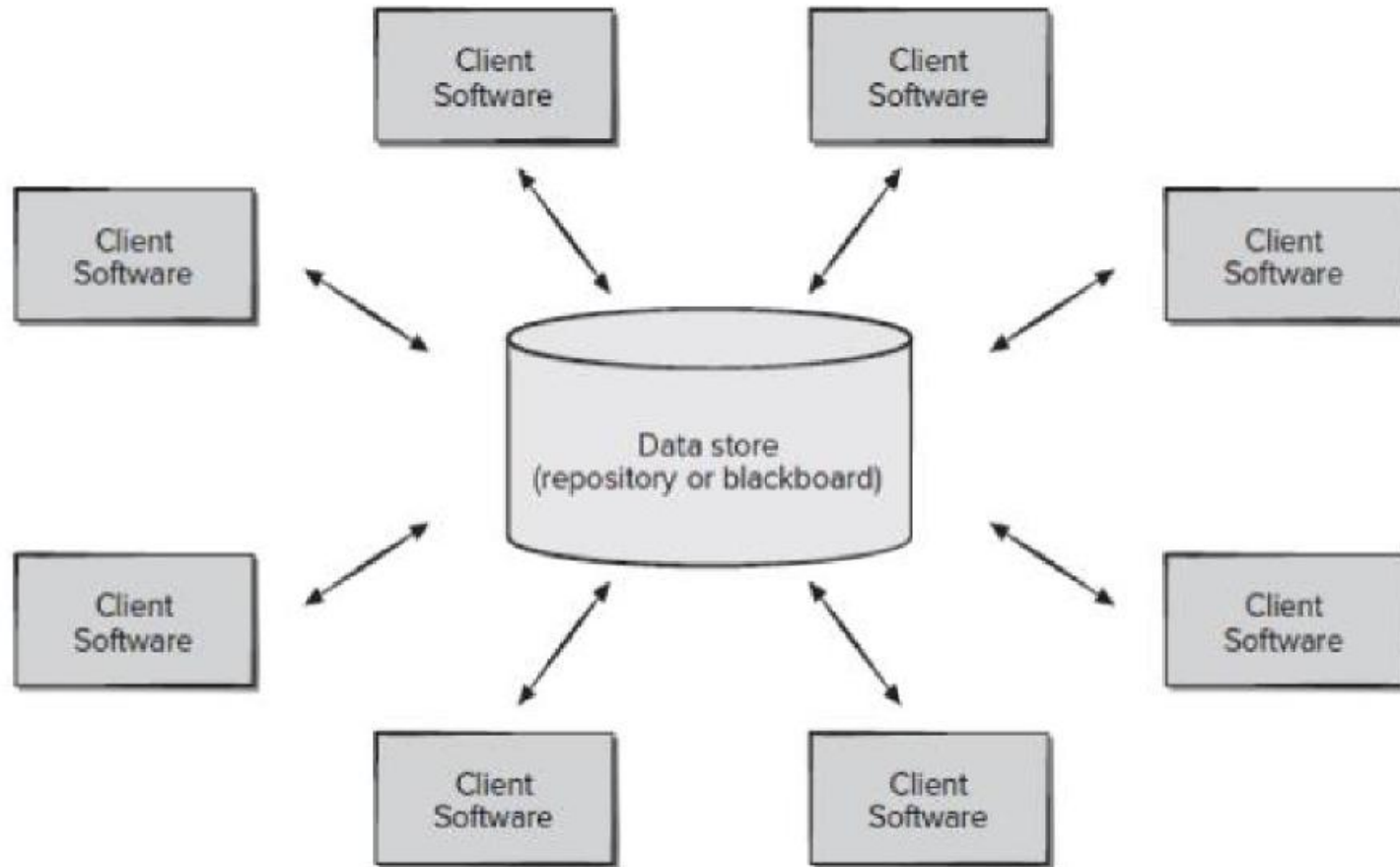
Layered Architecture



Data-Centered Architecture

- **A central data store** (repository) is accessed by all software components.
- **Components** (clients) read and write to the same shared database or file.
- **Commonly used** in banking, ERP, and web systems where many users share data.
- **Advantages:**
 - Components can work independently.
 - Promotes data consistency and long-term storage.
- **Disadvantages:**
 - The central repository can be a single point of failure.
 - Distribution across many computers is difficult.

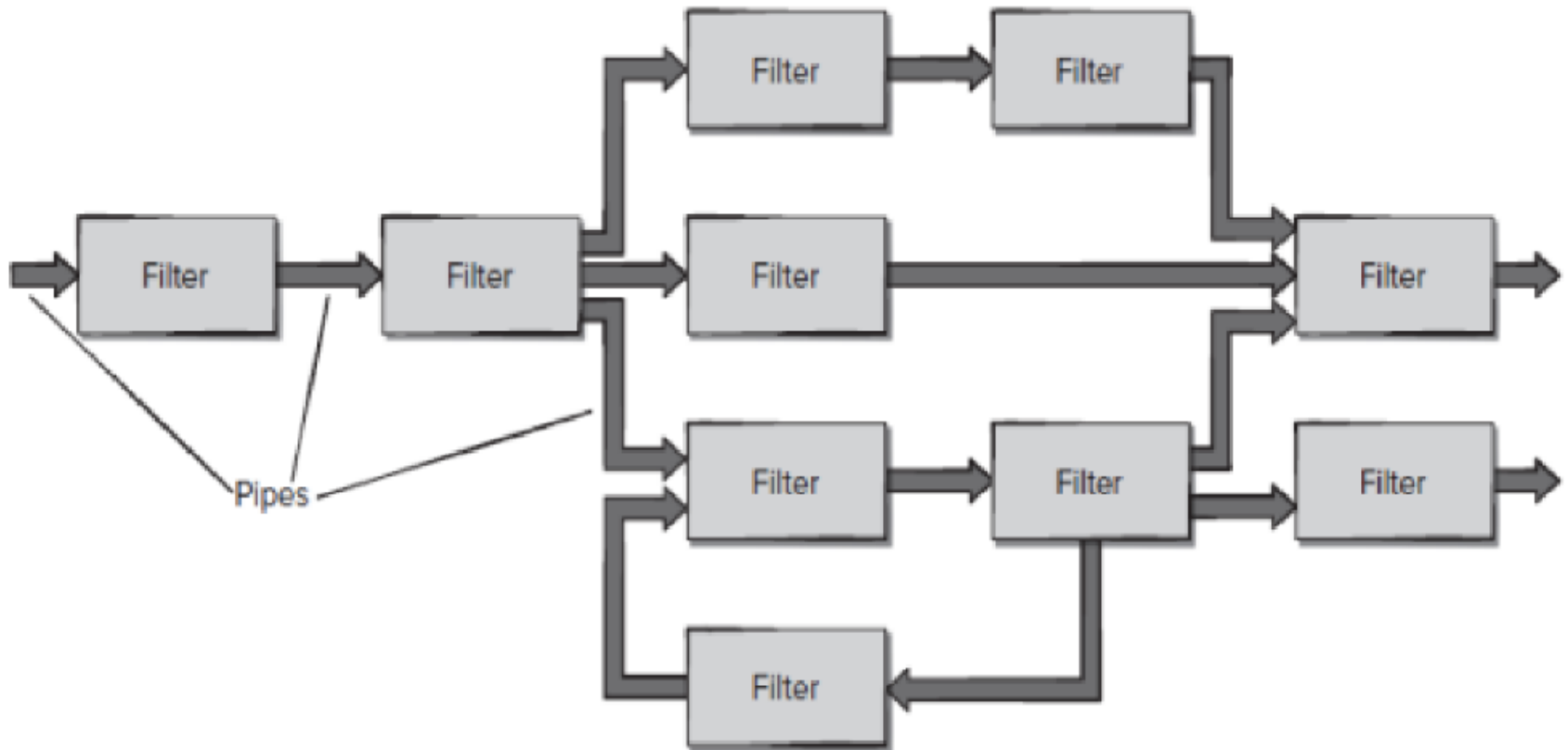
Data-Centered Architecture



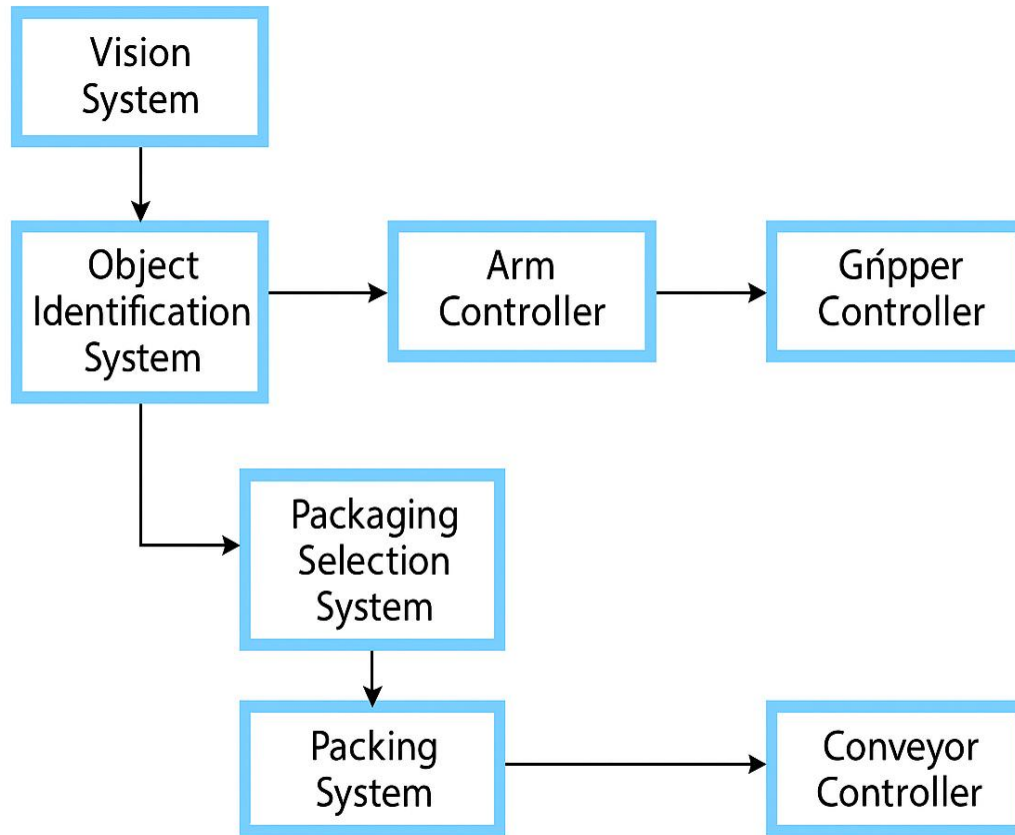
Data-Flow Architecture (Pipe and Filter)

- System is composed of filters (**processing steps**) connected by pipes (data flow).
- Each **filter** receives **input**, **transforms it**, and **sends output** to the **next filter**.
- **Pipe** is used for connecting filters and transferring data between them — often in data transformation, signal processing, or batch data processing systems.
- **Advantages:**
 - Filters are independent and reusable.
 - Easy to modify or add new filters.
- **Disadvantages:**
 - Not suitable for highly interactive or event-driven systems.
 - Overhead due to repeated data passing between filters.

Data-Flow Architecture



Architecture of a Packing Robot Control System (Exam)



The architecture of a packing robot control system

- The **packing robot control system** is designed to automate the process of identifying, selecting, and packaging different objects. It uses several interconnected components:
- The **Vision System** detects objects on a conveyor.
- The **Object Identification System** determines the type of object.
- The **Packaging Selection System** chooses the right packaging based on the object type.
- The **Arm and Gripper Controllers** move and handle the objects.
- The **Packing System** completes the packaging process.
- The **Conveyor Controller** transfers packaged objects to another conveyor.
- Each **box** in the diagram represents a component, and the **arrows** show the **flow of data and control signals** between components. The diagram illustrates how system components communicate to perform tasks efficiently.

Architectural design questions to consider include

- How the system will be distributed across processors?
- Which architectural style fits best?
- How components and subcomponents interact?
- What strategies control system operations?
- How the design meets functional and non-functional requirements?
- How it will be evaluated and documented?
- **In short sentences:**
 - The model presents a clear architectural view of a robotic system that integrates sensing, control, and packaging components to automate the packing process effectively.
 - A **component** is a functional unit of software that performs one major task and interacts with other components to form the complete system.

Issues in architectural design includes:

- Breaking the system into main components
- Allocation of functional responsibilities to components.
- Component Interfaces.
- 4. Each component should be efficient, lightweight, and reliable.
- 5. Communication and interaction between components.

* **Architectural design** adds structure and details that are missing from the interface level; however, the internal design of each component is completed later.

Detailed Design

- **Detailed design** is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and the data structures.
- **In short sentences:**
 - Focuses on the internal logic of each part.
 - Example: Code that calculates the total price of an order.

Cohesion in Software Design

- **Cohesion** measures how closely the tasks inside a software module are related.
A **highly cohesive** module performs one clear, specific function.
Low cohesion occurs when unrelated tasks are grouped together.
- Types of Cohesion (from low → high):
 1. **Coincidental** – unrelated tasks grouped randomly.
 2. **Logical** – tasks related by logic, not by function.
 3. **Temporal** – tasks executed in the same time frame.
 4. **Procedural** – tasks executed in a fixed sequence.
 5. **Communicational** – tasks using the same data area.
- **Goal:** Always design modules with high cohesion for clarity, reuse, and easier testing.

Coupling Between Modules

- Coupling tells us **how much one software module depends on another**.

When modules are **loosely connected**, they can work and be changed **independently**.

Low coupling → easier to understand, test, and maintain.

When they are **tightly connected**, a change in one module can **break or affect** the others.

High coupling → harder to modify; everything depends on everything.

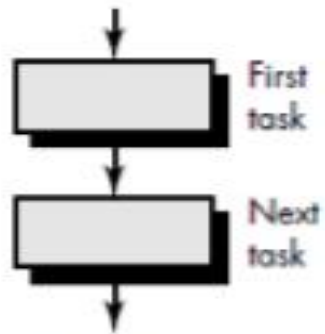
- **Types of Coupling (from low → high)**

1. No Direct Coupling – modules work separately and don't share any data. (Best case)
 2. Data Coupling – modules share only simple data (like numbers or text) through parameters.
 3. Stamp Coupling – modules share part of a data structure (e.g., one record of a list).
 4. Control Coupling – one module tells another what to do by sending control signals or flags.
 5. External Coupling – modules depend on external systems, APIs, or hardware devices.
 6. Common Coupling – multiple modules use the same global variables.
 7. Content Coupling – one module directly accesses or changes another module's code or data. (Worst case).
- **Goal:** Design software with low coupling and high cohesion so that modules stay independent, safe to modify, and easy to test.

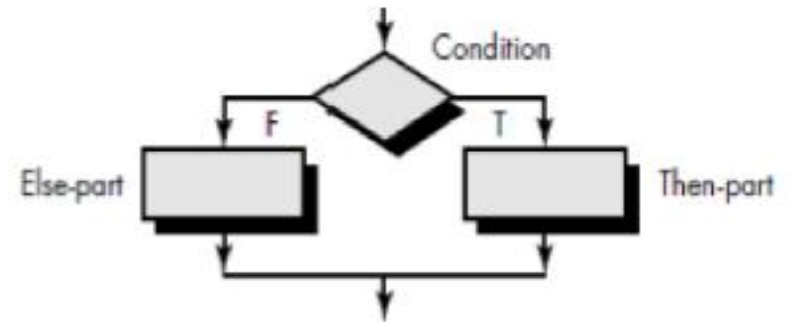
Structured Programming Constructs

- **Structured Programming** is a fundamental software design method based on three main control structures:
 - **Sequence:** steps are executed one after another (first → next).
 - **Condition (Selection):** a decision chooses which path to follow.
 - **Repetition (Loop):** actions are repeated until a condition is met.
- These three structures make code **readable, testable, and easy to maintain.**
- They can also be represented visually using **flowcharts** or **box diagrams.**

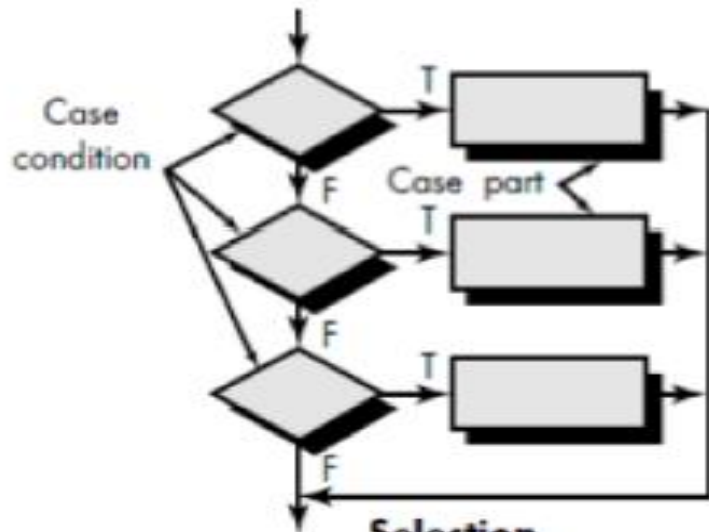
Flowchart Constructs



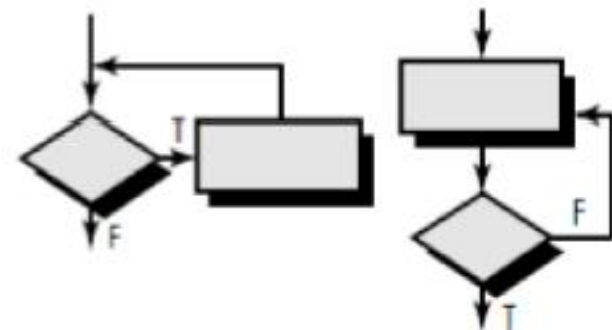
Sequence



If-then-else



Selection

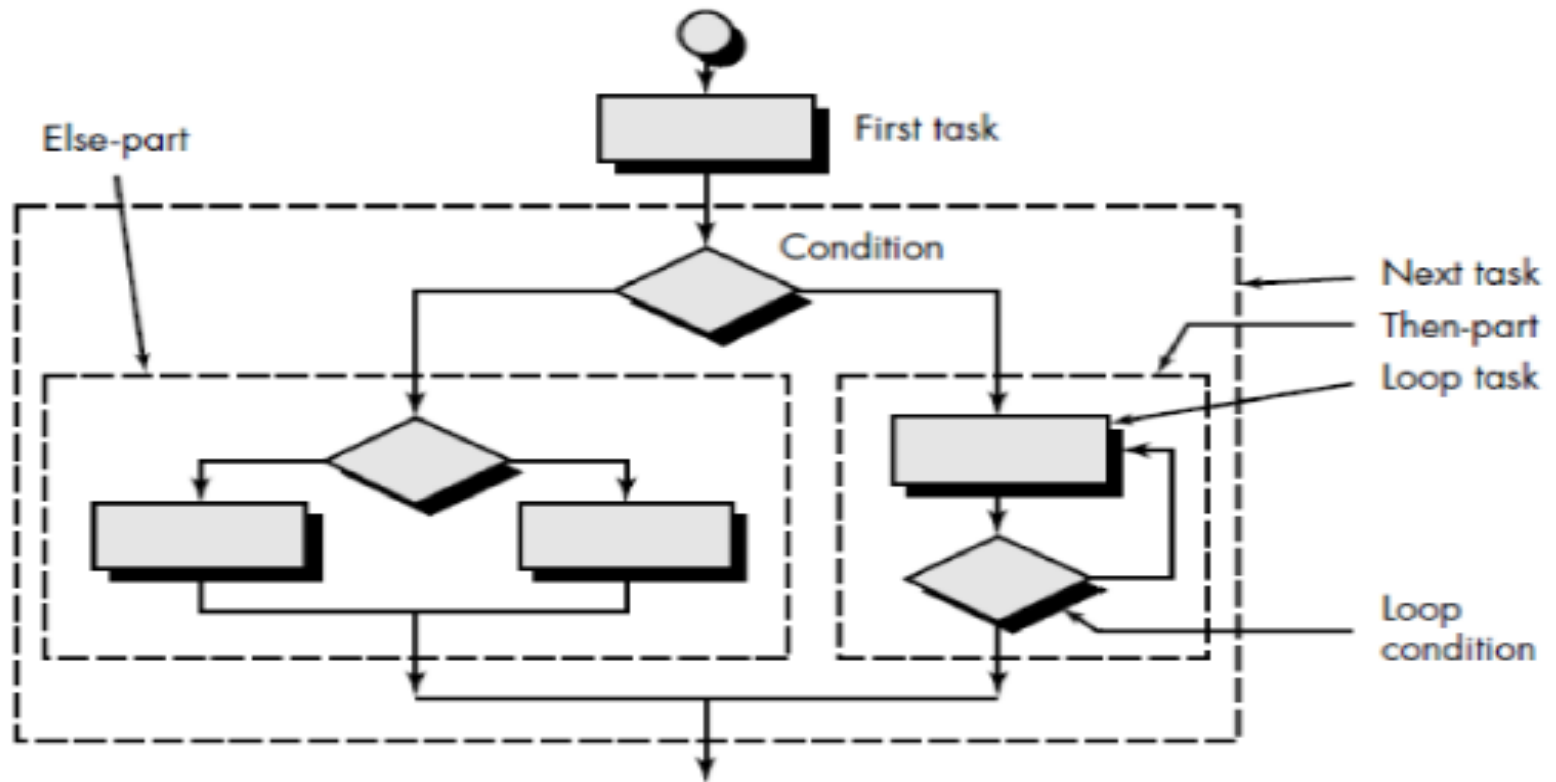


Do while

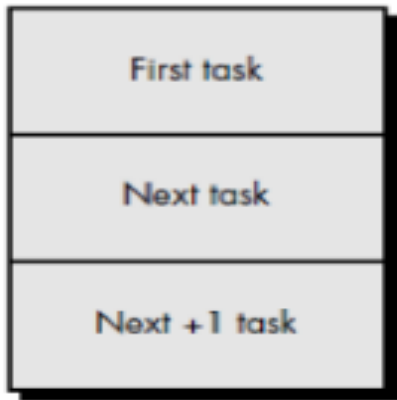
Repeat until

Repetition

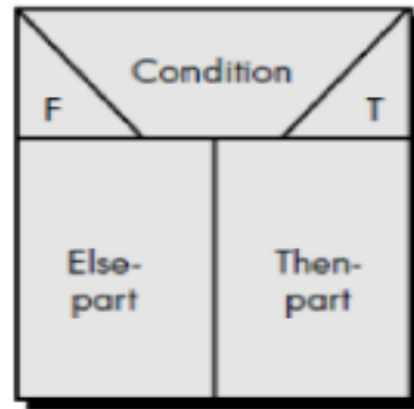
Nested Constructs



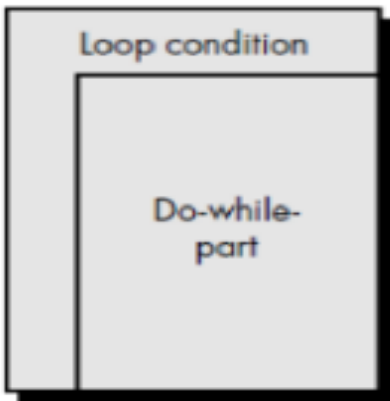
Box diagram constructs



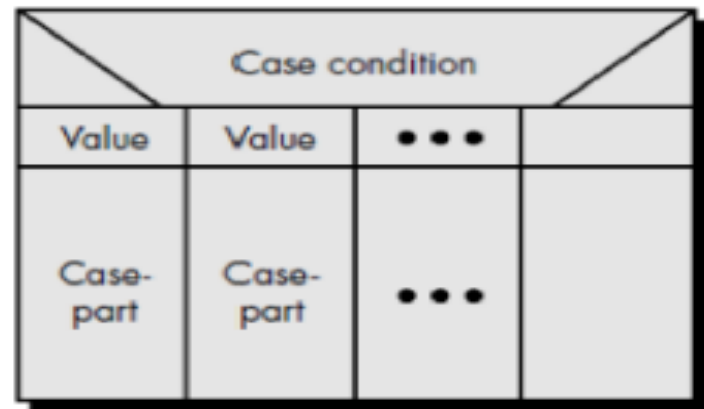
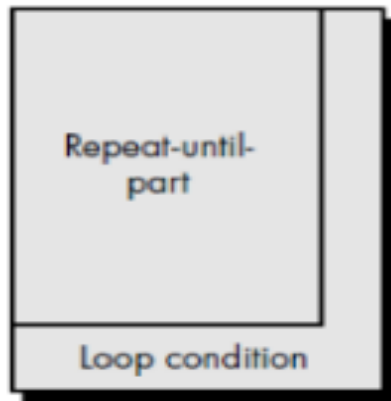
Sequence



If-then-else



Repetition



Selection

Decision Table

- **Definition**
- A **decision table** organizes complex logic into a **matrix of conditions and actions**.
- Each **column** (rule) represents a specific combination of conditions leading to an action.
- **Steps to Develop a Decision Table**
 1. List all **actions** that can occur in the procedure.
 2. List all **conditions/decisions** that affect these actions.
 3. **Match** each valid condition set with corresponding actions.
 4. Define **rules** showing which actions occur for which condition sets.
- **Purpose**
- Simplifies complex decision logic.
- Ensures all possible cases are considered.
- Helps developers and testers verify system behavior easily.

Decision Table

Conditions	Rules							
	1	2	3	4				n
Condition #1	✓			✓	✓			
Condition #2		✓		✓				
Condition #3			✓		✓			
Actions								
Action #1	✓			✓	✓			
Action #2		✓		✓				
Action #3			✓					
Action #4			✓	✓	✓			
Action #5	✓	✓			✓			

Decisions Table Structures

Conditions	Rules				
	1	2	3	4	5
Fixed rate acct.	T	T	F	F	F
Variable rate acct.	F	F	T	T	F
Consumption <100 kwh	T	F	T	F	
Consumption ≥100 kwh	F	T	F	T	
Actions					
Min. monthly charge	✓				
Schedule A billing		✓	✓		
Schedule B billing				✓	
Other treatment					✓

Resultant Decisions Table

The detailed design may include:

1. Decomposition of major system components into program units.
2. Allocation of functional responsibilities to units.
3. User interfaces.
4. Unit states and state changes.
5. Data and control interaction between units.
6. Data packaging and implementation, including issues of scope and visibility of program elements.
7. Algorithms and data structures.

Software Design Phases

- The software design process moves from requirements to a complete, well-structured system.

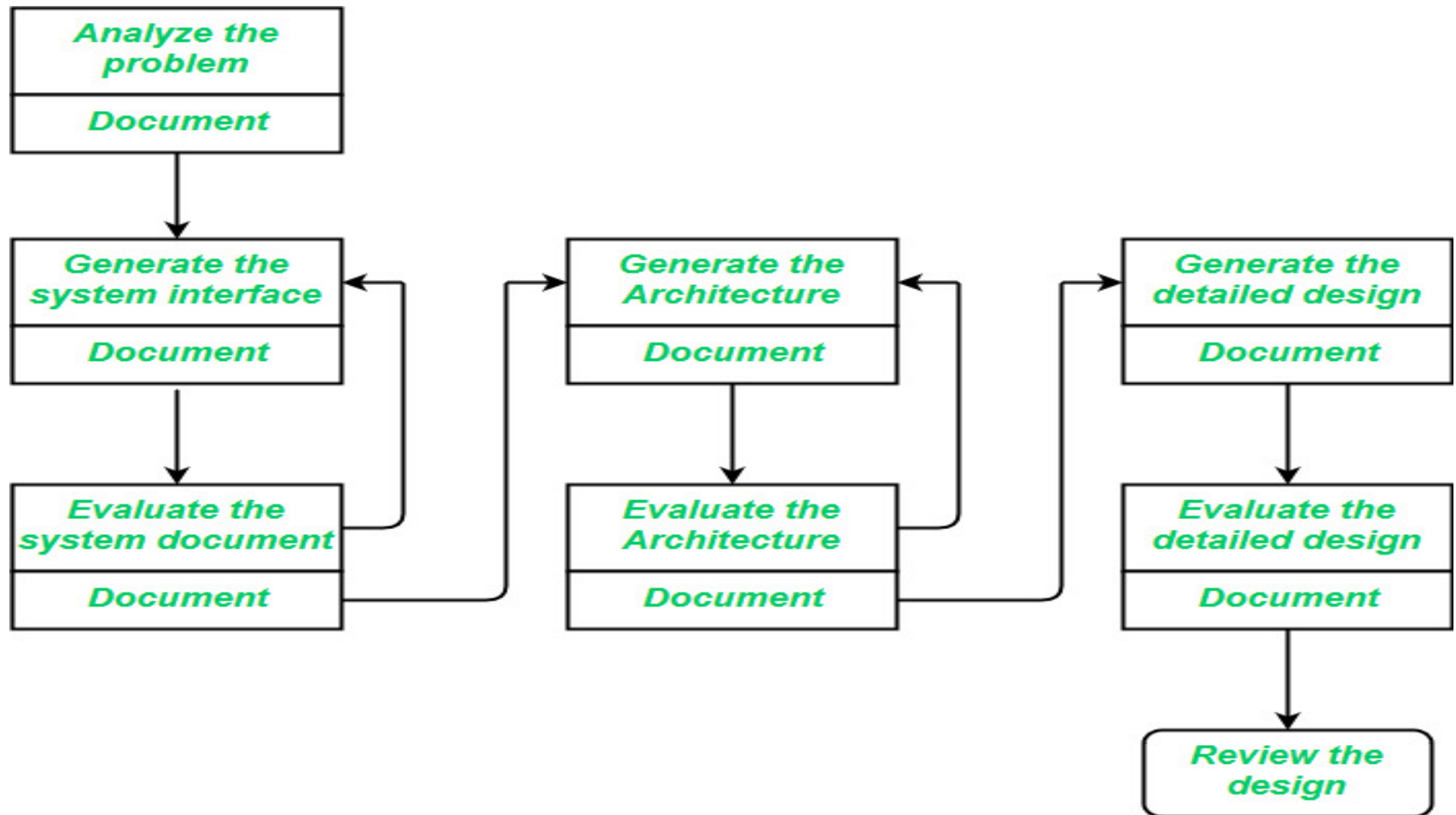
Software Design Phases



- **Understanding Project Requirements:** First, understand user needs, business goals, and challenges to form a solid design foundation.
- **Research, Analysis, and Planning:** The team collects data through interviews and surveys. This helps them understand user needs and design with users in mind.
- **Designing the Software:** The team creates wireframes, user stories, and flow diagrams. They build and refine prototypes based on user feedback.

- **Technical Design:** The team prepares a detailed technical document. It defines system components and how they will work together.
- **User Interface Design:** Focus on making the software easy and intuitive to use. Designers work on visuals, navigation, and user experience.
- **Prototyping:** Prototypes show how the system will look and work. They range from simple sketches to fully interactive models.

Elements of Software Design (Exam)



- **Architecture:** Defines structure and behavior, Shown using flowcharts and Keeps the system stable and easy to maintain.
- **Modules**
- Basic building blocks, Each does one task and Easy to build, test, and maintain.
- **Components:** Group of related modules and Keep code clean and flexible.
- **Interfaces:** Connection between components and Control how parts communicate.
- **Data:** Core of the system, Stores, shares, and manages information.

How Software Design Fits into the SDLC

Software design begins after requirements are complete and before development starts. In this phase, the team uses tools like wireframes and data flow diagrams to plan how the system will work. It acts as a roadmap showing how all components fit together before coding begins.

Design concepts

- **Architecture:**

Describes how software components are structured and how they work together to perform the required functions.

- **Abstraction:**

Hide complex details and show only key features, Makes the system easier to use and understand.

- **Modularity:**

Divide the software into small, independent parts,
Each part does one job and is easy to test or update.

- **Coupling:**

Keep connections between modules minimal,
Changing one part should not break others.

- **Anticipation of Change**

Design for future updates and new features.
Keeps the system flexible and easy to modify.

Design concepts

- **Simplicity**

Keep the design clean and clear.

Simple systems are faster and have fewer errors.

- **Completeness**

Include all required features, but avoid extras.

Ensure the system meets every needed function.

- **Information Hiding**

Each software module should hide its internal data and algorithms from other modules. This improves security, reduces complexity, and limits the impact of changes, making the software easier to maintain and update.

- **Functional Independence**

Software modules should perform one clear task with minimal interaction with other modules. It is achieved by High Cohesion and Low Coupling.

This makes modules easier to test, reuse, and maintain.

Design concepts

- **Stepwise Refinement**

A **top-down design approach** where complex software is broken into smaller, detailed steps. Starting from a high-level concept, each step adds more detail until the design is ready for implementation.

It ensures **clarity, structure, and systematic development**.

- **Refactoring**

Simplifies and reorganizes design/code without changing its behavior. Removes redundancy, unused elements, inefficient algorithms, or poor data structures.

Tools for Software Design

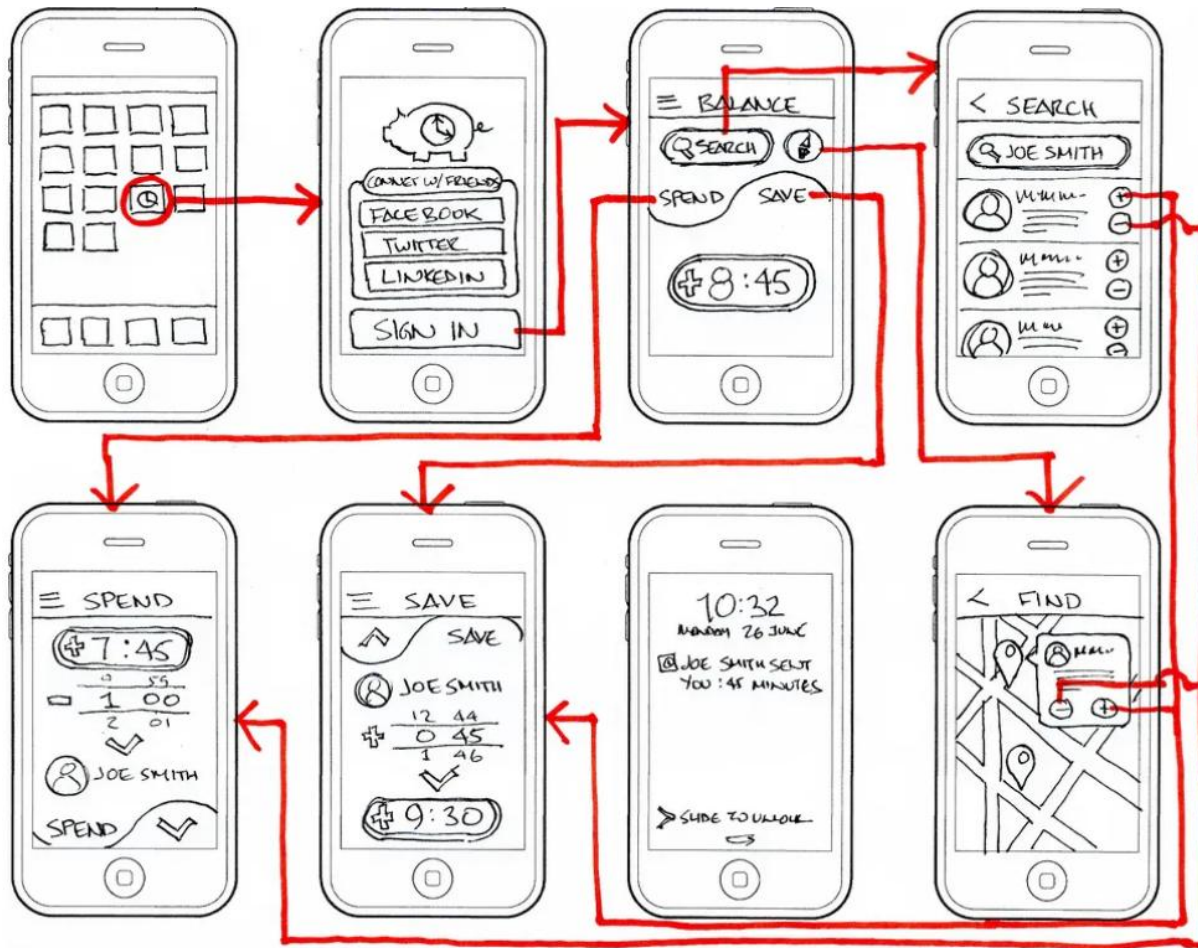
- There are many tools that make the process easier and more efficient, whether you're working on wireframes, prototypes, or documentation. Here are some popular tools:

1. Figma
2. Balsamiq
3. Axure RP
4. Sketch
5. InVision Studio

Wireframes and Visual blueprints

- **Wireframes:** are simple visual blueprints or sketches that show how a system's user interface (UI) will look and function before adding colors, images, or final design details.
(**Wireframes** focus on layout and structure, not decoration).
- **Visual blueprints** are diagrams that show a software system's design, structure, and behavior, serving as high-level plans to help developers and stakeholders share a clear understanding of project goals.

Wireframes



Software Design - Summary

Software Design – Key Takeaways (Main point to remember):

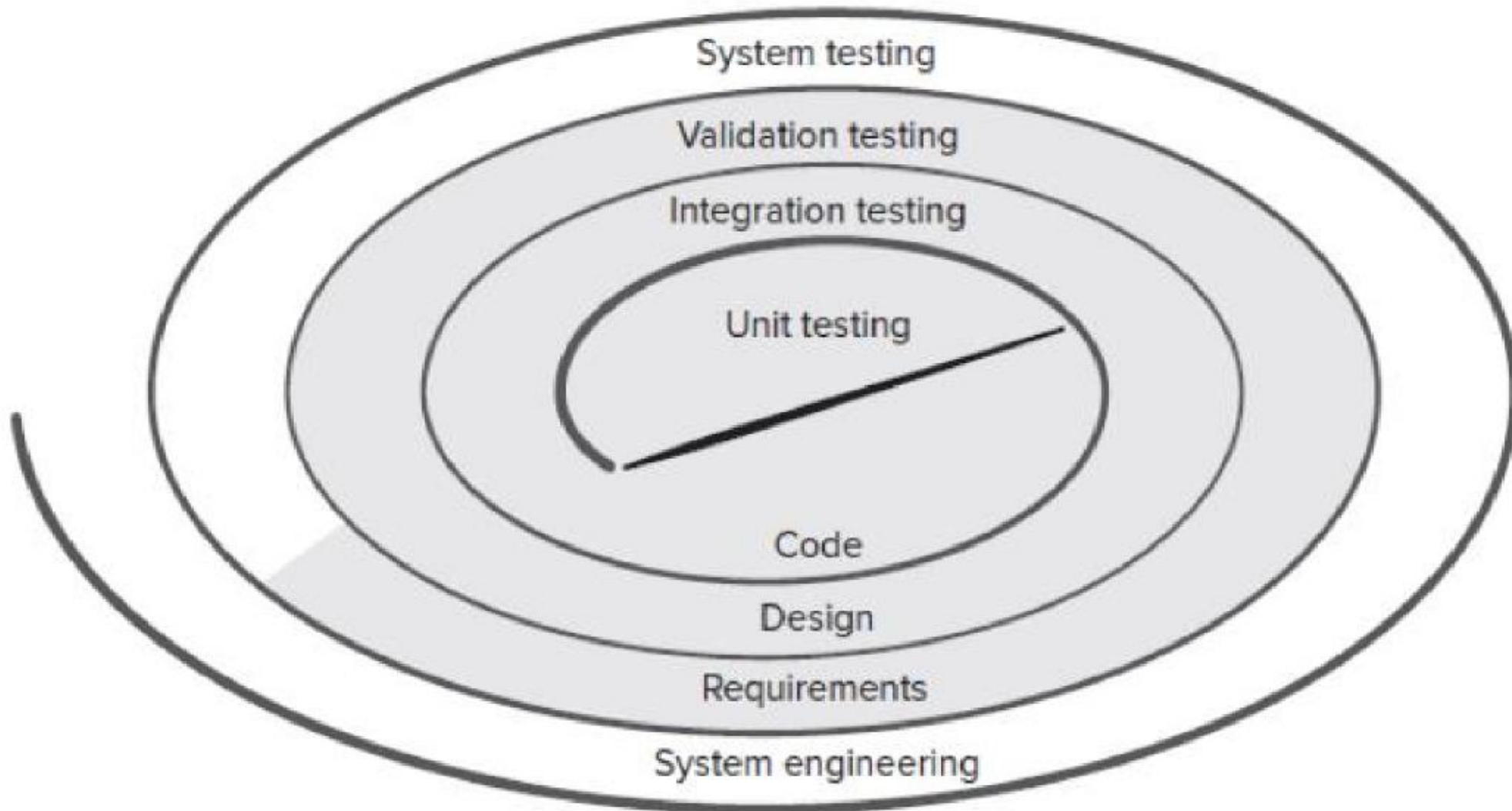
- 1- Clear structure (Architecture, Interface, Data).
- 2- High Cohesion + Low Coupling.
- 3- Structured Logic & Decision Tables ensure clarity.

Conclusion

- Software Design is an important part of the Software Development process, and it plays a major role in the clean and clear development of a product. Here we learned about the Software Design in detail. You can refer SDLC process for more information on Software development.

Thank you

Software Testing Strategy Diagram



Software Testing Strategy Diagram

- The diagram represents a **layered software testing strategy**.

It shows that testing progresses from the smallest software units to the complete operational system. Each outer layer covers a broader scope than the inner one.

Software Testing Strategy Diagram

1- Unit Testing

- This is the innermost layer.
- A unit is the smallest testable component of the software, such as a function, method, or class.
- Unit testing focuses on verifying that each unit behaves correctly in isolation, independent of other components.
- Its main objectives are to detect defects early, validate the correctness of the implemented logic, and reduce the cost of later rework.

Software Testing Strategy Diagram

2- Integration Testing

- Once individual units have been verified, they are combined, and their interactions are tested.
- Integration testing examines how modules exchange data and cooperate to provide higher-level functionality.
- The goal is to reveal defects that arise from incorrect interfaces, data mismatches, or misunderstandings between components.

Software Testing Strategy Diagram

4- Validation Testing

- At this level, the focus shifts from internal structure to **user and stakeholder requirements.**
- The central question becomes: Does the implemented system satisfy the specified requirements and user expectations?
- Validation testing often uses scenarios, use cases, and acceptance criteria derived directly from the requirements document.

Software Testing Strategy Diagram

5- System Testing

- This is the outermost layer, where the **entire system** is tested in an **environment** that is as close as possible to the real operational environment.
- System testing includes **performance**, **security**, **reliability**, **usability**, **compatibility**, and **recovery tests**.
- Its purpose is to ensure that the system operates as a coherent whole and is ready for deployment.

Software Testing Strategy Diagram

- The lower part of the diagram (Code → Design → Requirements → System Engineering) indicates that these testing activities are closely related to the development artifacts: unit tests relate to code, integration tests to design and component interactions, validation tests to requirements, and system tests to the overall engineered system.

Applying the Strategy to a Restaurant Application

Consider a **Restaurant Application** that supports online ordering, kitchen operations, delivery tracking, and management reporting.

Applying the Strategy to a Restaurant Application

1- Unit Testing in the Restaurant App

Example units: price calculation function, discount calculator, login validator, “add item to cart” method.

Unit tests verify, for instance, that **calculatorTotal ()** correctly computes the order amount, or that the login validator rejects invalid credentials.

Applying the Strategy to a Restaurant Application

- **Integration Testing in the Restaurant App**
- Typical integrations:
 - Customer interface ↔ Cart service
 - Cart service ↔ Payment gateway
 - Order service ↔ Kitchen display system
 - Order service ↔ Driver assignment module
- Integration testing verifies that an order created by the customer is correctly passed through these services, with correct data formats and states.

Applying the Strategy to a Restaurant Application

- **Validation Testing in the Restaurant App**
- Here we ask: Does the system fulfil what restaurant owners and customers requested in the requirements?
- Example checks:
 - Can a customer place and cancel an order as specified?
 - Are payment options (cash, card, wallet) supported as required?
 - Do restaurant staff receive clear and timely order information?
- These tests are usually expressed as acceptance scenarios derived from the requirements specification.

Applying the Strategy to a Restaurant Application

5- System Testing in the Restaurant App

1. The entire workflow is exercised end-to-end, e.g.:

Customer registers, browses the menu, places an order, pays, the kitchen prepares it, a driver is assigned, the order is delivered, and management reports are updated.

2. Non-functional aspects are also tested: performance under heavy load, security of customer data, resilience to failures, and usability of the interfaces.

Chapter Five

Software Evolution and Maintenance

- Software development does not end when a system is delivered; instead, it continues throughout the system's lifetime. Once deployed, a system must evolve to remain useful and relevant. This evolution is driven by **system change proposals**, which may arise from unimplemented requirements, new user needs, bug reports, or ideas from developers.

Chapter Five

Software Evolution and Maintenance

- The **software evolution process** includes several key activities:
 - 1.Change analysis** – studying the effect of proposed changes
 - 2.Release planning** – deciding which changes to implement in the next version
 - 3.Change implementation** – modifying specifications, design, and code
 - 4.System release** – delivering a new version to users

If changes are approved, a new release is planned and implemented. This process is iterative and repeats with each new set of change requests.

Chapter Five

Software Evolution and Maintenance

- **Software maintenance** refers to modifying a system after delivery. It is categorized into three main types:
 - **Fault repair (Corrective maintenance):** fixing errors and bugs
 - **Environmental adaptation (Adaptive maintenance):** adjusting the system to changes in hardware, operating systems, or platforms
 - **Functionality addition (Perfective maintenance):** adding or improving features due to business or organizational changes
- Finally, **software reengineering** is introduced as a way to improve legacy systems. It focuses on enhancing system structure, readability, and maintainability without changing its core functionality.

Chapter Five

Software Evolution and Maintenance

1. Software maintenance is performed (after) system delivery.
2. Fixing bugs is known as (corrective) maintenance.
3. Adapting software to a new operating system is (adaptive) maintenance.
4. Software evolution is an (iterative) process.

Requirements Validation Review

- **What is requirements validation, and why is it important in software engineering?**

Requirements validation is the process of checking gathered and documented requirements to ensure correctness. It is important because errors in the requirements document lead to costly rework when discovered during development or after deployment.

Requirements Validation Review

- **Why can incorrect requirements cause major problems later in a project?**

Because discovering requirement errors after development starts or after system deployment requires redesigning, re-coding, re-testing, and may affect the schedule and budget.

Requirements Validation Review

- **Which type of check ensures that no contradictory or conflicting requirements exist?**

Consistency checks.

- **A user requests a feature, but deeper analysis shows additional hidden functions are needed. Which validation check detects this?**

Validity checks.

Requirements Validation Review

- **What does a completeness check ensure in the requirements document?**

It ensures all required functions and constraints are correctly defined and included.

- **What does realism checking involve, and what two real-world constraints must be considered?**

It checks if requirements can actually be implemented using existing technology. It must consider **budget** and **schedule** constraints.

Requirements Validation Review

- **NOTE:** Verifiability means requirements should be testable and described in a way that allows writing tests.
- **Why must system requirements always be verifiable?**
To avoid disputes between customer and contractor and to ensure that every requirement can be tested after implementation.
- **How does requirements validation relate to requirements modeling?**
Validation checks if requirements are correct; modeling represents these validated requirements using different diagrams or models.

Software Validation

- **1. Software Testing & V&V**
- Software testing is the process of running a program with the goal of finding errors. It is part of the broader domain of **Verification and Validation (V&V)**.
- **Verification:** Ensures the software correctly implements specific functions.
“Are we building the product right?”
- **Validation:** Ensures the software meets customer needs and requirements.
“Are we building the right product?”

Software Validation

- **2. Attributes of a “Good” Test**

- A high-quality test should have:

1. **High probability of finding errors.**

2. **Non-redundancy**: avoid repeating the same test unnecessarily.

3. **Best of breed**: from similar tests, choose the one most likely to uncover errors.

4. **Balanced complexity**: not too simple and not overly complex; executed independently.

Software Validation

- **3. Software Quality**

Software quality is evaluated using three main groups of factors:

- **A- Product Operation Factors**

1- Correctness: How well the program satisfies requirements and mission goals.

2- Reliability: The ability of the program to perform its intended function accurately over time.

3- Usability: Effort required to learn, operate, and interpret the system.

4- Integrity: Protection against unauthorized access or misuse of data.

5- Efficiency: How well the program uses computing resources and code to carry out its function.

Software Validation

- **B. Product Revision Factors**

These describe how easy it is to modify or improve the software:

6. **Maintainability**: Effort required to find and fix errors.

7. **Flexibility**: Ease of modifying the software for new conditions.

8. **Testability**: Effort required to test the system and verify behavior.

Software Validation

- **C. Product Transition Factors**

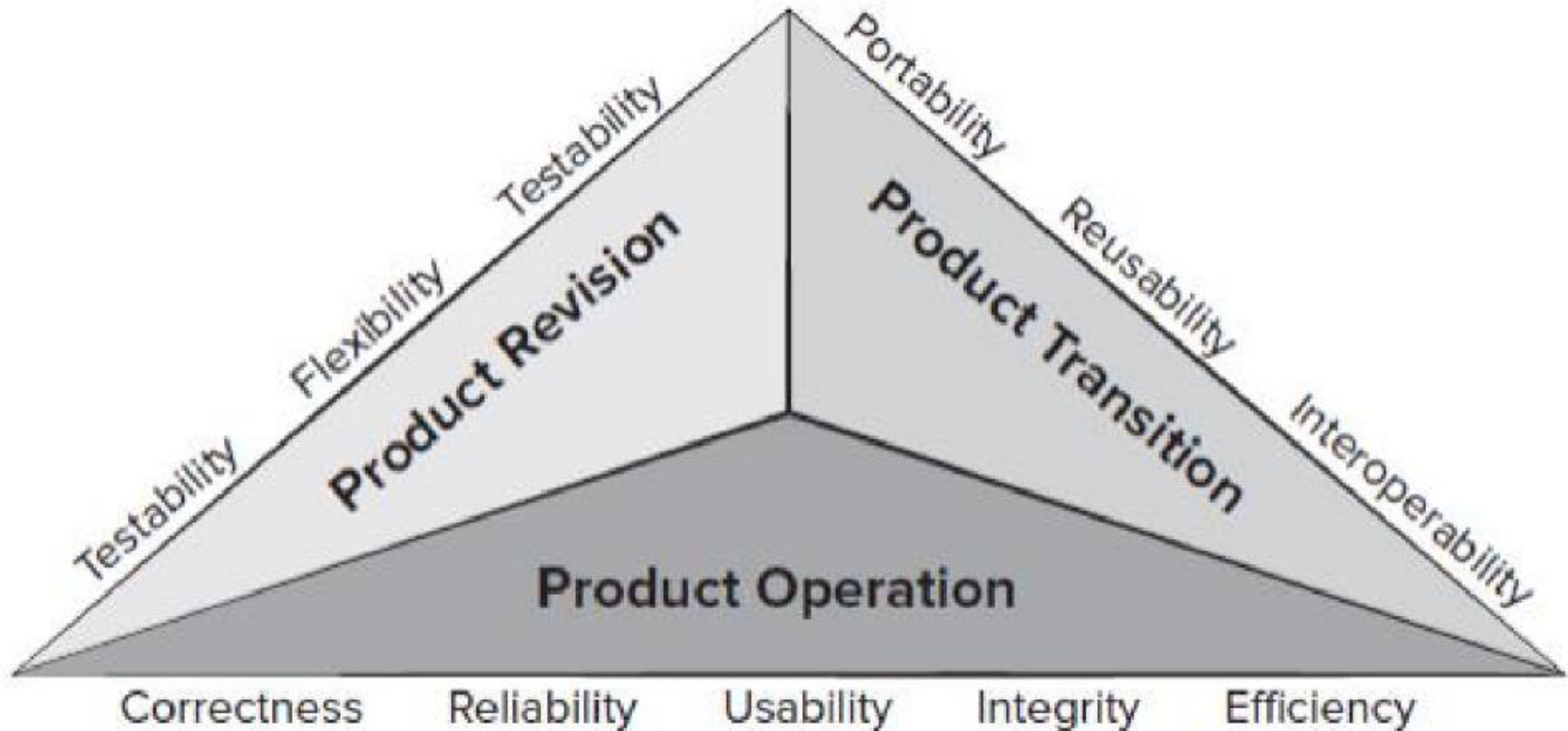
These describe how easily the software adapts to new environments:

9. Portability: Ease of moving the software to another platform.

10. Reusability: Ability to reuse parts of the system in other systems.

11. Interoperability: Ability of the software to work with other systems.

Quality Factors



Software Validation

- **2. Formal Technical Review (FTR)**
- A **Formal Technical Review** is a structured quality control activity performed by software engineers.
- **Objectives of FTR:**
 1. **Identify errors** in logic, function, or implementation.
 2. **Verify requirement compliance:** ensure the software meets documented needs.

Software Validation

- What are the three major categories of **software quality factors**?

Product Operation, Product Revision, Product Transition.

- What does integrity measure in software quality?

The extent to which unauthorized access to software or data is prevented.

- Which quality factor describes how much effort is needed to find and correct errors?

Maintainability.

- Why is portability important in software systems?

It measures the ease of moving a program between different hardware or software environments.

Software Validation

- **Reusability** refers to using the software in the exact same environment.

Correct: It refers to using the program (or parts of it) in *other* applications.

- **What does interoperability mean?**

The effort required to connect and operate software with other systems.

- **What is the main purpose of a Formal Technical Review (FTR)?**

To uncover errors and verify that the software meets its requirements.

Why is testability an important quality factor?

Because tests confirm that the software performs its intended function and meets quality standards.

Software Validation

- **A Formal Technical Review (FTR)** is a structured software quality assurance activity performed by software engineers to evaluate the correctness and completeness of software documents or code.
Its main goals are:
 - **Detect errors early** in function, logic, requirements, or implementation.
 - **Verify compliance** with system requirements.
 - **Ensure the software follows predefined standards** (coding, documentation, design).
 - **Promote uniformity** across the development team so all artifacts use consistent styles and formats.
 - **Improve project manageability**, reducing rework, cost, and schedule delays.
- FTR is important because it prevents defects from moving to later stages of development, where they become more expensive and harder to fix.

Software Validation

- **What is the main purpose of a Formal Technical Review (FTR)?**
To uncover errors and verify that the software meets its requirements.
- **Why is testability an important quality factor?**
Because tests confirm that the software performs its intended function and meets quality standards.

Software Validation

- **Which factor relates to ease of learning and operating a program?**

Usability.

- The ability of a system to work with another system is called **interoperability.**

- **Why is maintainability important?**

It shows how easy it is to find and fix errors in the program.

- **What is portability?**

The effort required to move the software to another hardware or software environment.

Software Validation

- **Objectives of a Formal Technical Review**
 - **1. Uncover Errors in Function, Logic, or Implementation**
 - FTR helps detect:
 - Functional mistakes
 - Logical inconsistencies
 - Incorrect implementation
 - Missing or ambiguous requirements
 - Misinterpretation of specifications
- This ensures the software model or code is technically sound.

Software Validation

2- Verify That the Software Meets Its Requirements

- An FTR checks whether:
 - The representation correctly reflects the documented requirements
 - The design aligns with functional and non-functional requirements
 - The implementation satisfies the intended system behavior
- This aligns product evolution with stakeholder expectations.

Software Validation

3- **Ensure Compliance With Predefined Standards**

- FTR ensures the software or document:
 - Follows organizational coding standards
 - Uses standard naming conventions
 - Applies architectural patterns correctly
 - Adheres to industry best practices and IEEE/ISO standards
- This improves maintainability and consistency.

Software Validation

4- **Achieve Uniformity Across the Development Team**

- Formal reviews enforce:
 - Standardized documentation style
 - Consistent design notations (UML, ERD, DFD, etc.)
 - Unified coding practices
- This reduces confusion and improves collaboration.

Software Validation

5- Make Projects More Manageable

- By discovering errors early and enforcing uniformity, FTR helps:
 - Reduce rework
 - Minimize schedule slippage
 - Improve quality of project planning
 - Enable more reliable cost estimation
- Overall, FTR increases project stability and predictability.

Agile Software Development – Software Engineering

- Agile Software Development is an **iterative, incremental, and adaptive approach** to building software that focuses on **early delivery of value, continuous customer involvement, and rapid response to change**. Unlike traditional plan-driven models, Agile accepts that requirements evolve over time and treats change as an opportunity rather than a problem.
- Agile is used because it enables teams to **deliver working software early and continuously**, allowing customers to see real progress, provide feedback, and influence development decisions. By **focusing on value-driven features**, Agile minimizes unnecessary work and reduces the risk of building unwanted functionality.

- Beyond a process, Agile represents a **mindset and cultural shift** that prioritizes **people, collaboration, transparency, and shared responsibility**. Teams are empowered to self-organize, communicate openly, and make decisions that improve product quality and team morale.
- **The Agile Software Development Process is executed through short, time-boxed iterations (sprints). Each iteration includes:**
 1. Requirements Gathering
 2. Planning
 3. Development
 4. Testing
 5. Deployment
 6. Maintenance

- Agile development is guided by the **four core values of the Agile Manifesto:**

1. Individuals and interactions over processes and tools
2. Working software over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan

- These values are reinforced by **12 principles** that emphasize early delivery, welcoming change, frequent releases, close collaboration, sustainable development pace, technical excellence, simplicity, and continuous improvement.

- **Common Agile practices** such as **Scrum, Kanban, Continuous Integration, Test-Driven Development, and Pair Programming** support frequent feedback, high code quality, and fast adaptation.
- **Agile offers major benefits including faster time-to-market, improved customer satisfaction, higher software quality, reduced risk, and increased team motivation.**

However, **it also presents challenges** such as **limited predictability, scope creep, dependency on customer availability, reduced documentation, and difficulties in large-scale projects.**

- **In conclusion, Agile is not a single method, but a framework of values and practices that creates an environment where effective software solutions can continuously emerge in fast-changing conditions.**

Advantages and Disadvantages of Agile Software Development.

- **Advantages:**
- Early and continuous delivery
- High customer satisfaction
- Flexibility to change
- Improved quality through continuous testing
- Strong team collaboration
- **Disadvantages:**
- Uncertain schedules and costs
- Risk of scope creep
- Dependence on customer involvement
- Less documentation
- Difficulties in large projects

Important Review

- **Agile mainly** emphasizes delivering working software and the ability to adapt to changing requirements.
- **Agile development delivers software** frequently in small, incremental releases.
- **Agile reduces project risk by** delivering software early, detecting issues quickly, and incorporating continuous customer feedback.
- **Agile can be difficult to apply in large organizations** because coordination and face-to-face communication become challenging, and effective decision-making often requires experienced teams.

Restaurant Application Requirements Document (SRS Document)

System Overview:

The restaurant app allows customers to browse meals, place orders, pay online, and track deliveries.

Actors:

Customer, Restaurant, Delivery Driver, System Administrator.

Functional Requirements:

1. The customer can register and log in to the app.
2. The customer can view the menu and search for meals.
3. The customer can add meals to the cart and make payments.
4. The restaurant receives and prepares the orders.
5. The driver delivers the order to the customer.
6. The system admin manages users and data.

Non-Functional Requirements:

1. The system should respond within 3 seconds for any request.
2. The app must ensure data security and user privacy.
3. The system should support at least 10,000 concurrent users.
4. The interface should be simple and intuitive.

Constraints:

The app must be compatible with both Android and iOS devices.

If the software is running locally (offline), it must also work on standard desktop systems without requiring internet access.

General Requirements Document Template (SRS Document)

System Overview:

This section provides a summary of the system's purpose and main functionality.

Actors:

List all actors who will interact with the system (e.g., user, admin, external system).

Functional Requirements:

Describe what the system should do — its core functionalities and behaviors.

Non-Functional Requirements:

Define performance, usability, reliability, and security requirements.

Constraints:

List any hardware, software, or policy limitations that affect the project.